

DATA 260 Homework 2 Report

Sreeram Achutuni

SJSU ID: 019151808

Section 0: Config values

SID4	1808
PORT_BASE	8008
PREFIX	s1808
SEED	1808
VERIFY_SEED	261808
DOMAIN_ID	0, Campus course catalogue and enrolment
Hardware	Intel Core i7-11390H, 16GB RAM, integrated Iris Xe graphics (no dedicated GPU)
Local model	qwen2.5:1.5b-instruct (Ollama), used instead of the qwen3:8b default since this laptop has no GPU and that model was too slow for the amount of runs this homework needed
Repo	https://github.com/sriram2930/Data260-1808
Commit tagged as hw2	7ae65ed5058241c70c44fca8bd3a5800ce95db02

Part 1: HTML and CSS

The assignment wanted the HW1 course list and form to still be usable at a 375px screen width, and to have some visible loading, empty and error state instead of the page just doing nothing or breaking.

I checked the 375px width using Chrome dev tools (device toolbar, width set to 375). At that width everything stacks into one column and nothing gets cut off or needs horizontal scrolling, the search box and the course cards and the add form all still work normally.

245

Computer Science

ml

machine learning

Edit

Delete (highest id)

Add a Course

Course Code

245

Course Title

ml

Your Email

achutuni.sriram@gmail.com

Course Description

Machine learning course

Department

Computer Science

☐ I agree to the terms and conditions.

Loading...

DATA-260 · HW2 · Domain 0 · PREFIX s1808

Home page at 375px width, before adding anything.

For the loading state I did not build anything with a real spinner talking to a database, since this app just keeps everything in memory. What I did instead is make the submit buttons switch their text to "Loading..." and disable themselves right when you click, before the redirect actually

happens. It is a small thing but it is a real state change on the screen, not just a static page. Below is what it looks like mid submit while adding a course with code 245 and title ml:

Campus Course Catalogue & Enrolment

Add, edit, remove, and search course listings.

Search

Search

DATA-260

Data Science & AI

Big Data Technologies and Systems

Design and operation of data pipelines feeding ML systems.

Edit

CS-146

Computer Science

Data Structures and Algorithms

Core data structures, algorithm design, and complexity analysis.

Edit

BUS-181

Business Analytics

Business Analytics Fundamentals

Statistical and analytical techniques for business decision making.

Edit

Delete (highest id)

Add a Course

Course Code

Course Title

Add form showing the Loading state right after clicking submit.

The empty state shows up when a search does not match any course. I typed "abcd" into the search box on purpose, since that is not part of any course code or title in the list, and got the "No courses match your search" message instead of a blank list:

Campus Course Catalogue & Enrolment

Add, edit, remove, and search course listings.

Search

✕ Search

No courses match your search.

Add a Course

Course Code

Course code, e.g. DATA-260

Course Title

Course title, e.g. Big Data Technologies and

Your Email

e.g. jane.doe@sjsu.edu

Course Description

Describe the course content, topics covered, and learning outcomes...

Department

Computer Science

☐ I agree to the terms and conditions.

Submit Course Listing

Search with no matching results, empty state message shown.

The error state was supposed to be a red banner at the top telling you Course Code and Course Title are required. When I tested this earlier with curl it worked exactly like that. But when I actually tried it from the browser by leaving both fields blank and clicking submit, the browser itself blocked the submit with its own little tooltip ("Please fill out this field") because I still had the required attribute on those two inputs in the HTML. So my banner never even got a chance to show up here, the browser caught it first. I am leaving this in as is instead of quietly fixing it, since it is a real thing I found while testing rather than something I planned:

Search

No courses match your search.

Add a Course

Course Code

Course Title

!

Please fill out this field.

Your Email

Course Description

Describe the course content, topics covered, and learning outcomes...

Department

Computer Science

☐ I agree to the terms and conditions.

Submit Course Listing

DATA-260 · HW2 · Domain 0 · PREFIX s1808

Trying to submit with blank required fields, browser's own validation kicks in first.

Relevant CSS

The card layout rules:

```
.course-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.course-card {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 1rem 1.1rem;
}

.course-card .row1 {
  display: flex;
  justify-content: space-between;
  align-items: baseline;
  gap: 0.5rem;
  flex-wrap: wrap;
}

.course-card .code {
  font-weight: 700;
  font-size: 1rem;
}
```

More card rules, for the department pill, title, description and action buttons:

```
.course-card .dept {
  font-size: 0.78rem;
  color: var(--muted);
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 999px;
  padding: 0.1rem 0.55rem;
  white-space: nowrap;
}

.course-card .title {
  margin: 0.15rem 0 0.4rem;
  font-size: 0.95rem;
}

.course-card .desc {
  font-size: 0.85rem;
  color: var(--muted);
  margin: 0 0 0.6rem;
}

.course-card .actions {
  display: flex;
  gap: 0.5rem;
  flex-wrap: wrap;
}
```

The empty and error placeholder styling, plus the loading spinner:

```

.state-placeholder {
  text-align: center;
  padding: 1.75rem 1rem;
  border: 1px dashed var(--border);
  border-radius: 10px;
  color: var(--muted);
  font-size: 0.92rem;
  background: var(--card);
}

.state-placeholder.is-error {
  border-color: var(--error-border);
  background: var(--error-bg);
  color: var(--error-text);
}

.spinner {
  display: inline-block;
  width: 0.9rem;
  height: 0.9rem;
  border: 2px solid var(--border);
  border-top-color: var(--accent);
  border-radius: 50%;
  animation: spin 0.7s linear infinite;
  vertical-align: -2px;
  margin-right: 0.4rem;
}

```

And the actual responsive rule, the part that matters most for the 375px question, is this media query near the bottom of the stylesheet:

```

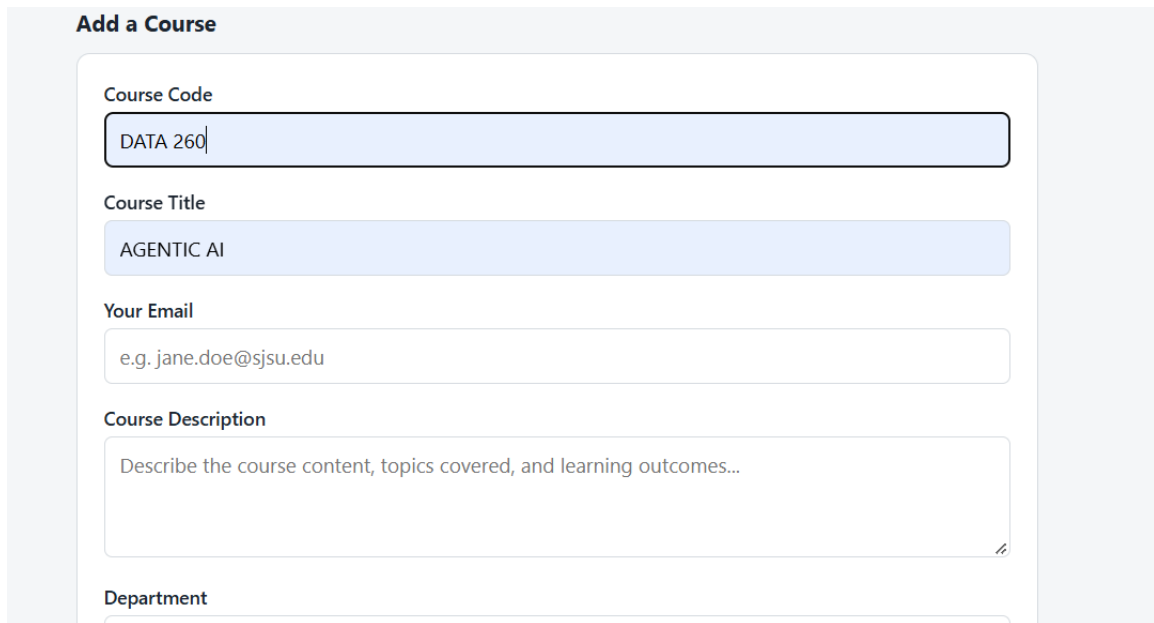
and to tighten padding slightly on very narrow phones. */
@media (max-width: 400px) {
  body { padding: 1rem 0.65rem 2.5rem; }
  h1 { font-size: 1.4rem; }
  .course-card .actions a,
  .course-card .actions button { flex: 1 1 auto; text-align: center; }
}

```

Part 2: FastAPI backend

This is the backend behind the form. Adding, updating and deleting all go through normal HTML form submits that redirect back to the home page afterward (a 303 redirect), not through JavaScript fetch calls. Search is the one exception, that one does use fetch so the list can update without reloading the whole page.

To test adding a course I filled in Course Code as "DATA 260" and Course Title as "AGENTIC AI" and left everything else blank on purpose, just to see if it would still go through with only those two filled in:



The image shows a web form titled "Add a Course". It contains five input fields: "Course Code" (text), "Course Title" (text), "Your Email" (text), "Course Description" (text area), and "Department" (text). The "Course Code" field contains the text "DATA 260". The "Course Title" field contains the text "AGENTIC AI". The "Your Email" field contains the placeholder text "e.g. jane.doe@sjsu.edu". The "Course Description" field contains the placeholder text "Describe the course content, topics covered, and learning outcomes...". The "Department" field is empty. The form is styled with a light blue background and rounded corners.

Add form filled in with a new course code and title.

After submitting, it shows up at the bottom of the list, filed under Computer Science since I never touched the department dropdown:

Search

Search

DATA-260

Data Science & AI

Big Data Technologies and Systems

Design and operation of data pipelines feeding ML systems.

Edit

CS-146

Computer Science

Data Structures and Algorithms

Core data structures, algorithm design, and complexity analysis.

Edit

BUS-181

Business Analytics

Business Analytics Fundamentals

Statistical and analytical techniques for business decision making.

Edit

DATA 260

Computer Science

AGENTIC AI

Edit

Delete (highest id)

Home page after the add, new course now in the list.

I then added a second, similar course (title "AGENTIC AI and Distributed systems") just so there would be a 5th record and the highest id delete button would have something obvious to remove:

DATA-260 Big Data Technologies and Systems Design and operation of data pipelines feeding ML systems. <a>Edit	Data Science & AI
CS-146 Data Structures and Algorithms Core data structures, algorithm design, and complexity analysis. <a>Edit	Computer Science
BUS-181 Business Analytics Fundamentals Statistical and analytical techniques for business decision making. <a>Edit	Business Analytics
DATA 260 AGENTIC AI <a>Edit	Computer Science
DATA 260 AGENTIC AI and Distributed systems <a>Edit <a>Delete (highest id)	Computer Science

List with a 5th record added, Delete (highest id) button now showing on it.

Clicking that Delete button removes the highest id record and drops you back on the home page with a small confirmation banner:

Campus Course Catalogue & Enrolment

Add, edit, remove, and search course listings.

Deleted course #5.

Search

DATA-260

Data Science & AI

Big Data Technologies and Systems

Design and operation of data pipelines feeding ML systems.

CS-146

Computer Science

Data Structures and Algorithms

Core data structures, algorithm design, and complexity analysis.

BUS-181

Business Analytics

Business Analytics Fundamentals

Statistical and analytical techniques for business decision making.

DATA 260

Computer Science

AGENTIC AI

After deleting course #5, back to 4 records, confirmation banner visible.

Search for "data" pulls in DATA-260 and the DATA 260 I added, plus CS-146 too, since "Data Structures" is in that course's title. So it is matching on both the course code and the title field, not just one of them:

Deleted course #5.

Search

data × Search

DATA-260 Data Science & AI

Big Data Technologies and Systems

Design and operation of data pipelines feeding ML systems.

Edit

CS-146 Computer Science

Data Structures and Algorithms

Core data structures, algorithm design, and complexity analysis.

Edit

DATA 260 Computer Science

AGENTIC AI

Edit Delete (highest id)

Search results for the query 'data'..

Relevant backend code

Adding a course:

```
@app.post("/courses")
def create_course(
    courseCode: str = Form(""),
    courseTitle: str = Form(""),
    submitterEmail: str = Form(""),
    description: str = Form(""),
    department: str = Form(DEPARTMENTS[0]),
    agreeTerms: Optional[str] = Form(None),
):
    if not courseCode.strip() or not courseTitle.strip():
        return RedirectResponse(
            url="/?error=Course+Code+and+Course+Title+are+required.", status_code=303
        )
    rec = dict(
        id=_new_id(),
        courseCode=courseCode.strip(),
        courseTitle=courseTitle.strip(),
        submitterEmail=submitterEmail.strip(),
        description=description.strip(),
        department=department,
        agreeTerms=bool(agreeTerms),
    )
    COURSES.append(rec)
    return RedirectResponse(url="/?success=Course+added.", status_code=303)
```

Loading the edit form for one record:

```

@app.get("/courses/{course_id}/edit")
def edit_course_form(request: Request, course_id: int):
    rec = _find(course_id)
    if rec is None:
        return RedirectResponse(url="/?error=Record+not+found.", status_code=303)
    return templates.TemplateResponse(
        request, "edit.html", {"course": rec, "departments": DEPARTMENTS}
    )

```

Updating a record by id, this is the route that is meant to update id 1:

```

@app.post("/courses/{course_id}/update")
def update_course(
    course_id: int,
    courseCode: str = Form(""),
    courseTitle: str = Form(""),
    submitterEmail: str = Form(""),
    description: str = Form(""),
    department: str = Form(DEPARTMENTS[0]),
    agreeTerms: Optional[str] = Form(None),
):
    rec = _find(course_id)
    if rec is None:
        return RedirectResponse(url="/?error=Record+not+found.", status_code=303)
    if not courseCode.strip() or not courseTitle.strip():
        return RedirectResponse(
            url="/?error=Course+Code+and+Course+Title+are+required.", status_code=303
        )
    rec.update(
        courseCode=courseCode.strip(),
        courseTitle=courseTitle.strip(),
        submitterEmail=submitterEmail.strip(),
        description=description.strip(),
        (function) agreeTerms: bool
        agreeTerms=bool(agreeTerms),
    )
    return RedirectResponse(url="/?success=Course+updated.", status_code=303)

```

Deleting the highest id record, and the app entry point:

```

@app.post("/courses/delete-highest")
def delete_highest():
    global COURSES
    hid = _highest_id()
    if hid is None:
        return RedirectResponse(url="/?error=No+records+to+delete.", status_code=303)
    COURSES = [c for c in COURSES if c["id"] != hid]
    return RedirectResponse(url=f"/?success=Deleted+course+%23{hid}.", status_code=303)

if __name__ == "__main__":
    import uvicorn

    uvicorn.run("main:app", host="0.0.0.0", port=PORT_BASE, reload=False)

```

The JSON endpoint the search box's fetch call hits:

```

@app.get("/api/courses")
def api_courses(q: str = ""):
    rows = [c for c in COURSES if _matches(c, q)] if q else COURSES
    return rows

```

Part 3: Stateful agent graph

This part takes the Planner and Reviewer from HW1 and puts them into an actual LangGraph graph instead of just calling one function after another. There is a Supervisor node that does not do any of the actual work itself, it just sits between Planner and Reviewer and a routing function decides where to go next based on whether there is a proposal yet, whether it passed validation, and whether the Reviewer approved it.

A normal run looks like this, Planner produces something, Reviewer approves it on the first try, done:

```
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808> .venv\Scripts\python.exe Part-2\agent_graph.py --input-file reports\hw02\cases\schema_input.json --turn-ceiling 10
---NODE: Supervisor (turn_count=0, ceiling=10)---
---NODE: Planner (attempt 1/10)---
-> valid proposal: ['machine learning', 'data systems', 'pipelines']
---NODE: Supervisor (turn_count=1, ceiling=10)---
---NODE: Reviewer---
-> approved=True issues=[]
---NODE: Supervisor (turn_count=1, ceiling=10)---

=== FINAL STATE ===
{
  "title": "DATA-260: Data Systems for ML",
  "content": "This course covers data pipelines for machine learning: schema design, streaming ingestion, and monitoring for data drift.",
  "email": "",
  "strict": true,
  "task": "tag and summarize",
  "planner_proposal": {
    "tags": [
      "machine learning",
      "data systems",
      "pipelines"
    ],
    "summary": "This course focuses on data systems for machine learning, including schema design, streaming ingestion, and monitoring for data drift."
  },
  "reviewer_feedback": {
    "approved": true,
    "issues": []
  },
  "turn_count": 1,
  "turn_ceiling": 10,
  "validation_error": null,
  "force_reviewer_issues": false
}

Outcome: valid_first_attempt
Latency: 9726.6 ms
```

Normal run, approved on the first Planner attempt.

To actually prove the correction loop works and is not just theoretical, I ran it again with a flag that forces the Reviewer to reject no matter what, so I could watch it send the task back to the Planner instead of just trusting that the code would do that if it needed to:

```

Outcome: valid first attempt
Latency: 9726.6 ms
--NODE: Supervisor (turn_count=0, ceiling=3)---ata260-1808>
--NODE: Planner (attempt 1/3)---
-> valid proposal: ['data systems', 'machine learning pipelines', 'data quality']
--NODE: Supervisor (turn_count=1, ceiling=3)---
--NODE: Reviewer---
-> (forced) rejecting to test the correction loop
--NODE: Supervisor (turn_count=1, ceiling=3)---
--NODE: Planner (attempt 2/3)---
-> INVALID: 1 validation error for PlannerTags
tags
  Value error, tag 'ML' must be 3-39 characters, got 2 [type=value_error, input_value=['ML', 'Data Pipelines', 'Machine Learning'], input_type=list]
  for further information visit https://errors.pydantic.dev/2.13/v/value_error
--NODE: Supervisor (turn_count=2, ceiling=3)---
--NODE: Planner (attempt 3/3)---
-> valid proposal: ['Data Pipelines', 'Machine Learning', 'Data Systems']
--NODE: Supervisor (turn_count=3, ceiling=3)---
--NODE: Reviewer---
-> (forced) rejecting to test the correction loop
--NODE: Supervisor (turn_count=3, ceiling=3)---

=== FINAL STATE ===
{
  "title": "DATA-260: Data Systems for ML",
  "content": "This course covers data pipelines for machine learning: schema design, streaming ingestion, and monitoring for data drift.",
  "email": "",
  "strict": true,
  "task": "tag and summarize",
  "planner_proposal": null,
  "reviewer_feedback": {
    "approved": false,
    "issues": [
      "forced rejection for testing"
    ]
  },
  "turn_count": 3,
  "turn_ceiling": 3,
  "validation_error": null,
  "force_reviewer_issues": true
}

Outcome: hit turn ceiling
Latency: 8235.1 ms

```

Reviewer forced to always reject, graph keeps looping back to the Planner until the turn ceiling is hit.

The state and the nodes

The shared state every node reads from and writes back to:

```

# -----
class AgentState(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: Optional[Dict[str, Any]]
    reviewer_feedback: Optional[Dict[str, Any]]
    turn_count: int
    turn_ceiling: int
    validation_error: Optional[str]
    force_reviewer_issues: bool

```

The Planner node. It builds the prompt, adds on whatever went wrong last time (either a validation error or a Reviewer rejection) so the model has something concrete to fix, then tries to parse and validate the response:

```

def planner_node(state: AgentState) -> Dict[str, Any]:
    turn = state.get("turn_count", 0) + 1
    print(f"---NODE: Planner (attempt {turn}/{state['turn_ceiling']}---")

    client: ModelClient = state["llm"]
    user_block = f"TITLE: {state['title']}\n\nCONTENT: {state['content']}"

    if state.get("validation_error"):
        user_block += (
            f"\n\nYour previous attempt was INVALID: {state['validation_error']}\n"
            "Fix this and respond again with ONLY the corrected JSON object."
        )

    feedback = state.get("reviewer_feedback")
    if feedback and not feedback.get("approved", True):
        user_block += (
            f"\n\nThe Reviewer rejected your previous proposal for these reasons: "
            f"{feedback.get('issues', [])}\nAddress them and respond again with ONLY "
            "the corrected JSON object."
        )

    response = client.complete(
        [
            {"role": "system", "content": PLANNER_SYSTEM_PROMPT},
            {"role": "user", "content": user_block},
        ],
        format="json",
    )

    try:
        parsed = json.loads(response["content"])
        validated = PlannerTags(**parsed)
        print(f" -> valid proposal: {validated.tags}")
        return {
            "planner_proposal": validated.model_dump(),
            "validation_error": None,
            "turn_count": turn,
        }
    except (json.JSONDecodeError, ValidationError, TypeError) as e:
        print(f" -> INVALID: {e}")
        return {
            "planner_proposal": None,
            "validation_error": str(e),
            "turn_count": turn,
        }

```

The Reviewer node, plus the forced rejection test hook mentioned above:

```

def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print("---NODE: Reviewer---")

    if state.get("force_reviewer_issues"):
        # Test hook for the Part 3 Step 6 correction-loop demonstration: force
        # a rejection (no LLM call needed) and confirm the graph routes back
        # to the Planner.
        print(" -> (forced) rejecting to test the correction loop")
        return {
            "reviewer_feedback": {"approved": False, "issues": ["forced rejection for testing"]},
            "planner_proposal": None,
        }

    client: ModelClient = state["llm"]
    proposal = state["planner_proposal"]
    user_block = (
        f"TITLE: {state['title']}\n\nCONTENT: {state['content']}\n\n"
        f"PLANNER TAGS: {json.dumps(proposal['tags'])}\n\n"
        f"PLANNER SUMMARY: {proposal['summary']}"
    )
    response = client.complete(
        [
            {"role": "system", "content": REVIEWER_SYSTEM_PROMPT},
            {"role": "user", "content": user_block},
        ],
        format="json",
    )

    try:
        feedback = json.loads(response["content"])
        approved = bool(feedback.get("approved"))
    except json.JSONDecodeError:
        # Treat unparsable Reviewer output as a rejection so the graph keeps
        # making forward progress (another Planner retry) instead of crashing.
        feedback = {"approved": False, "issues": ["reviewer output was not valid JSON"]}
        approved = False

    print(f" -> approved={approved} issues={feedback.get('issues', [])}")
    if approved:
        return {"reviewer_feedback": feedback}
    return {"reviewer_feedback": feedback, "planner_proposal": None}

```

The Supervisor node, which mostly just exists to sit in between the other two and print where things are:

```

def supervisor_node(state: AgentState) -> Dict[str, Any]:
    print(f"---NODE: Supervisor (turn_count={state.get('turn_count', 0)}), "
          f"ceiling={state['turn_ceiling']}---")
    return {}

```

Part 4: Output schema and loop safety

The routing function that actually decides planner vs reviewer vs stop:

```
def router_logic(state: AgentState) -> str:
    feedback = state.get("reviewer_feedback")
    if feedback and feedback.get("approved"):
        return END
    if state.get("planner_proposal") is not None and not state.get("validation_error"):
        # A fresh, schema-valid, not-yet-reviewed proposal exists -- always
        # let the Reviewer weigh in on it, even if this was the ceiling-th
        # Planner attempt.
        return "reviewer"
    if state.get("turn_count", 0) >= state.get("turn_ceiling", DEFAULT_TURN_CEILING):
        return END
    return "planner"
```

Below is one of the runs from the 30 run experiment. This particular one needed a few retries, the Reviewer rejected the first valid proposal for being too vague, then a couple of attempts failed the tag validation entirely (wrong number of tags in the list), before it eventually landed on something both schema valid and Reviewer approved:

```
lucency@140722 ~$
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808> .venv\Scripts\python.exe Part-2\agent_graph.py --input-file reports\hw02\cases\schema_input.json --turn-ceiling 10
---NODE: Supervisor (turn_count=0, ceiling=10)---
---NODE: Planner (attempt 1/10)---
-> valid proposal: ['data pipelines', 'Machine Learning', 'Stream Ingestion']
---NODE: Supervisor (turn_count=1, ceiling=10)---
---NODE: Reviewer---
-> approved=False issues=[]
---NODE: Supervisor (turn_count=1, ceiling=10)---
---NODE: Planner (attempt 2/10)---
-> INVALID: 1 validation error for PlannerTags
tags
Value error, must have exactly 3 tags, got 5 [type=value_error, input_value=['data pipelines', 'machi...data drift monitoring'], input_type=list]
For further information visit https://errors.pydantic.dev/2.13/v/value_error
---NODE: Supervisor (turn_count=2, ceiling=10)---
---NODE: Planner (attempt 3/10)---
-> INVALID: 1 validation error for PlannerTags
tags
Value error, must have exactly 3 tags, got 4 [type=value_error, input_value=['data pipelines', 'machi...ngestion', 'monitoring'], input_type=list]
For further information visit https://errors.pydantic.dev/2.13/v/value_error
---NODE: Supervisor (turn_count=3, ceiling=10)---
---NODE: Planner (attempt 4/10)---
-> valid proposal: ['data pipelines', 'machine learning', 'data monitoring']
---NODE: Supervisor (turn_count=4, ceiling=10)---
---NODE: Reviewer---
-> approved=False issues=["Use specific, non-vague tags like 'schema design', 'streaming ingestion', 'data drift monitoring'."]
---NODE: Supervisor (turn_count=4, ceiling=10)---
---NODE: Planner (attempt 5/10)---
-> valid proposal: ['schema design', 'streaming ingestion', 'data drift monitoring']
---NODE: Supervisor (turn_count=5, ceiling=10)---
---NODE: Reviewer---
-> approved=True issues=[]
---NODE: Supervisor (turn_count=5, ceiling=10)---

=== FINAL STATE ===
{
  "title": "DATA-260: Data Systems for ML",
  "content": "This course covers data pipelines for machine learning: schema design, streaming ingestion, and monitoring for data drift.",
  "email": "",
  "strict": true,
  "task": "tag_and_summarize",
  "planner_proposal": {
    "tags": [
      "schema design",
      "streaming ingestion",
      "data drift monitoring"
    ],
    "summary": "This course focuses on data pipelines for machine learning: schema design, streaming ingestion, and monitoring for data drift."
  },
  "reviewer_feedback": {
    "approved": true,
    "issues": []
  },
}
```

```

=== FINAL STATE ===
{
  "title": "DATA-260: Data Systems for ML",
  "content": "This course covers data pipelines for machine learning: schema design, streaming ingestion, and monitoring for data drift.",
  "email": "",
  "strict": true,
  "task": "tag_and_summarize",
  "planner_proposal": {
    "tags": [
      "schema design",
      "streaming ingestion",
      "data drift monitoring"
    ],
    "summary": "This course focuses on data pipelines for machine learning: schema design, streaming ingestion, and monitoring for data drift."
  },
  "reviewer_feedback": {
    "approved": true,
    "issues": []
  },
  "turn_count": 5,
  "turn_ceiling": 10,
  "validation_error": null,
  "force_reviewer_issues": false
}

Outcome: valid_after_2plus_retries
Latency: 17279.8 ms
(.venv) PS C:\Users\User\Desktop\Data_260_HW-1\data260-1808>

```

One run from the classification experiment, ends up in the valid_after_2plus_retries bucket.

And the summary block after all 75 runs (30 for classification, 20 plus 20 for the ceiling comparison, 5 adversarial):

```

=== SUMMARY ===
{
  "classification_30_runs": {
    "valid_first_attempt": {
      "count": 14,
      "mean_latency_ms": 4682.4
    },
    "valid_after_1_retry": {
      "count": 8,
      "mean_latency_ms": 8366.1
    },
    "valid_after_2plus_retries": {
      "count": 7,
      "mean_latency_ms": 14656.6
    },
    "hit_turn_ceiling": {
      "count": 1,
      "mean_latency_ms": 28392.1
    }
  },
  "ceiling_comparison": {
    "2": {
      "n": 20,
      "completion_rate": 0.7,
      "mean_latency_ms": 6109.4
    },
    "10": {
      "n": 20,
      "completion_rate": 1.0,
      "mean_latency_ms": 9714.8
    }
  },
  "adversarial": {
    "n": 5,
    "hit_ceiling_rate": 0.8
  }
}

Raw rows (CSV): C:\Users\User\Desktop\Data_260_HW-1\data260-1808\reports\hw02\raw\schema_experiment_runs.csv
Raw rows (JSON): C:\Users\User\Desktop\Data_260_HW-1\data260-1808\reports\hw02\raw\schema_experiment_runs.json
Summary: C:\Users\User\Desktop\Data_260_HW-1\data260-1808\reports\hw02\raw\schema_experiment_summary.json

=== finished 2026-09-13 18:44:15 ===

```

Final summary printed after the full experiment script finished.

Results

Outcome over 30 runs	Count	Mean latency (ms)
Valid first attempt	14	4682.4
Valid after 1 retry	8	8366.1
Valid after 2+ retries	7	14656.6
Hit turn ceiling	1	28392.1

Turn ceiling	Completion rate	Mean latency (ms)
2	70%	6109.4
10	100%	9714.8

Between the two, I would go with a turn ceiling of 10 for anything real. Ceiling 2 finishes about 3.6 seconds faster on average, but it also fails to ever get an approved answer 30% of the time, which is a pretty rough failure rate for something a user is actually waiting on. Ceiling 10 costs more time on average but got every single run to a valid, approved result.

Adversarial input

For the adversarial case I reused the longer, denser course description from HW1 (the one with schema design, batch and streaming ingestion, feature stores, versioning, drift monitoring, all packed into one paragraph). Running it 5 times, it hit the turn ceiling 4 out of 5 times.

My best guess for why: there is just too much going on in that description for a 25 word summary to cover, so the model keeps writing a longer summary that covers more of it instead of a short one, even after being told exactly how many words over it was. It is not that the model cannot count to 25, it seems more like it is choosing to keep the content over following the limit. A fix I would try if I had more time is not just repeating the word count error back to it, but telling it something more mechanical like "cut this down to one clause" or giving it a hard example of the right length, since just repeating the number does not seem to be enough on its own.